

Database 2

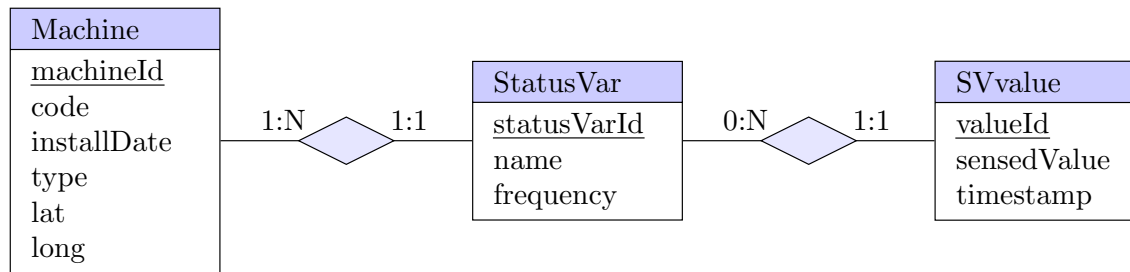
JPA exercise

Exam on Jun 27 2022

Solutions

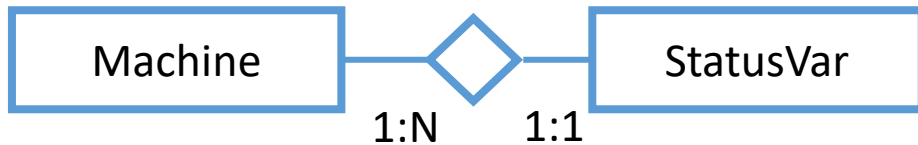
Specifications

- An application manages the data collected via an IoT architecture from industrial machines for anomaly detection purposes. The user can inspect a list of machines and see the status variables that are collected by the IoT sensors associated with the machine. A machine has a code (int), a type (string), an installation date (date), and a geolocation (two float values for latitude and longitude). A status variable has a name (string) and a sampling frequency (e.g., one value per minute, hour, day) coded as an integer value. Each status variable is associated to multiple values read by an IoT sensor. A sensed value is a floating point number associated with a timestamp (DateTime).



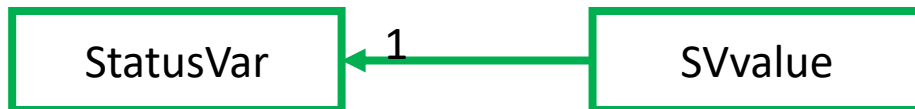
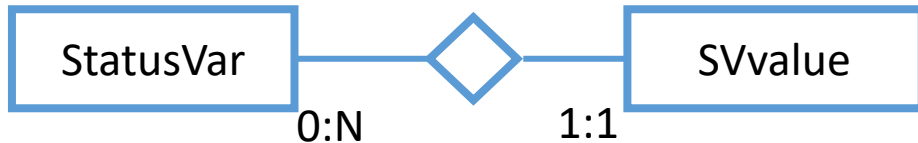
- The user can select one machine, then select one variable and open a histogram that shows the sensed values over time.
- Machines are in the order of hundreds, variables per machine are in the order of tens, sensed values per variable are in the order of thousands.
- Show the JPA entities (with all their attributes) that map the domain objects of the conceptual model, taking into account the above mentioned access paths of the application and data cardinalities. When designing the annotations for the relationships, specify the owner side of the relationship, the mapped-by attribute, the fetch policy and the cascading policies you consider more appropriate to support the access required by the web application. Comment on the design choices you have made.

Relationship machine variables



- Machine → StatusVar
@OneToMany
 - Necessary to browse the StatusVars of a Machine
 - Owner = StatusVar
 - FetchType is EAGER (StatusVars per Machine are in the order of tens)
 - Remove can be cascaded
- StatusVar → Machine
@ManyToOne
 - Not necessary but mapped for consistency
 - FetchType can be EAGER (one Machine per StatusVar)
 - No cascade is needed

Relationship value of status



- `StatusVar` → `SVvalue`
@OneToMany

- needed to show all the `SVvalue`s of a `StatusVar`
- Owner = `SVvalue`
- FetchType can be LAZY (`SVvalue`s per `StatusVar`s are in order of thousands)
- Remove can be cascaded

- `SVvalue` → `StatusVar`
@ManyToOne

- Not necessary but mapped for consistency
- FetchType can be EAGER (one Machine per `StatusVar`)
- No cascade is needed

Entity Machine

```
@Entity
public class Machine implements Serializable {
    @Id @GeneratedValue(strategy = GenerationType.AUTO)
    private int machineId;
    private int code;
    @Temporal(TemporalType.DATE)
    private Date installDate;
    private String type;
    private float lat;
    private float long;

    @OneToMany(mappedBy="machine", fetch = FetchType.EAGER,
        cascade = CascadeType.REMOVE)
        private List<StatusVar> statusVars;

    // getters setters and constructors
    // . . .
```

Entity StatusVar

```
@Entity
public class StatusVar implements Serializable {
    @Id @GeneratedValue(strategy = GenerationType.AUTO)
    private int statusVarId;
    private String name;
    private int frequency;

    @ManyToOne(fetch = FetchType.EAGER) // Eager can be omitted, default value
        @JoinColumn(name="machine") //owner of the relation
    private Machine machine;

    //LAZY can be omitted, default FetchType for collections
    @OneToMany(mappedBy="statusVar", fetch = FetchType.LAZY,
        cascade = CascadeType.REMOVE)
    private List<SVvalue> values;

    // getters setters and constructors
    // . . .
```

Entity SVvalue

```
@Entity
public class SVvalue implements Serializable {
    @Id @GeneratedValue(strategy = GenerationType.AUTO)
    private int valueId;
    private int sensedValue;
    @Temporal(TemporalType.TIMESTAMP)
    private Date timestamp;

    @ManyToOne(fetch = FetchType.EAGER) // Eager can be omitted, default
        @JoinColumn(name="statusVar") //owner of the relation
    private StatusVar statusVar;

    // getters setters and constructors
    // . . .
```